

1. Why Data Engineers Use Python

Python helps Data Engineers work with both:

- Data
- The complete process around the data

Common uses of Python in Data Engineering:

- Reading files
- Calling APIs
- Validating data
- Automating repetitive tasks
- Moving or renaming files
- Connecting different systems
- Handling failures
- Generating logs
- Building data pipelines

Important Point

SQL mainly works with data inside databases.

Python can manage files, APIs, validations, automation and pipeline logic.

2. Basic Data Pipeline Flow

- Source System
- ↓
- Extract Data
- ↓
- Validate Data
- ↓
- Transform Data
- ↓
- Load Data
- ↓
- Logs and Alerts

Python can be used at almost every stage of this flow.

3. Common Data Sources

A Data Engineer may receive data from:

- CSV files
 - JSON files
 - APIs
 - Databases
 - Cloud storage
 - Third-party applications
-

4. Python for API Extraction

Python can call an API and collect data.

- `import requests`
-
- `response = requests.get(`
- `"https://example.com/customers",`
- `timeout=30`
- `)`
-
- `customers = response.json()`

Key Idea

- `requests.get()` calls the API.
 - `timeout=30` prevents the request from waiting indefinitely.
 - `response.json()` reads the JSON response.
-

5. Python for File Validation

Python can validate incoming files before processing them.

- `file_name = "customers_2026_07_30.csv"`
-
- `if file_name.endswith(".csv"):`
- `print("Valid CSV file")`
- `else:`
- `print("Unsupported file format")`

This logic can be used to reject unsupported files before starting the pipeline.

6. Record-Count Validation

One common pipeline validation is comparing source and target record counts.

- `source_count = 1000`
- `target_count = 975`
-
- `rejected_count = source_count - target_count`

Calculation

- `Rejected Records = Source Count - Target Count`
 - `1000 - 975 = 25`
-

7. Assigning Pipeline Status

- `if source_count == target_count:`
- `pipeline_status = "SUCCESS"`
- `else:`
- `pipeline_status = "VALIDATION_FAILED"`

Logic

- Same source and target count → SUCCESS
- Different source and target count → VALIDATION_FAILED

A failed validation means the pipeline requires investigation.

8. Complete Validation Example

- pipeline_name = "customer_daily_load"
- source_name = "customer_api"
-
- source_count = 1000
- target_count = 975
-
- rejected_count = source_count - target_count
-
- if source_count == target_count:
- pipeline_status = "SUCCESS"
- else:
- pipeline_status = "VALIDATION_FAILED"
-
- print("Pipeline Execution Summary")
- print("-----")
- print(f"Pipeline Name: {pipeline_name}")
- print(f"Source Name: {source_name}")
- print(f"Source Records: {source_count}")
- print(f"Target Records: {target_count}")
- print(f"Rejected Records: {rejected_count}")
- print(f"Pipeline Status: {pipeline_status}")

Output

- Pipeline Execution Summary
- -----
- Pipeline Name: customer_daily_load
- Source Name: customer_api
- Source Records: 1000
- Target Records: 975
- Rejected Records: 25

- Pipeline Status: VALIDATION_FAILED

9. Python Concepts Used

Concept	Usage
Variable	Stores pipeline information
String	Stores names and status
Integer	Stores record counts
Arithmetic	Calculates rejected records
Comparison	Compares source and target
Condition	Assigns pipeline status
f-String	Displays formatted output

10. f-Strings

f-strings provide a clean way to display variables.

- `print(f'Source Records: {source_count}')`

The variable inside `{ }` is replaced with its actual value.

11. Python vs SQL

SQL

Used for:

- Filtering data
- Joining tables
- Aggregating data
- Querying databases
- Transforming stored data

Python

Used for:

- Reading files
- Calling APIs
- Moving files
- Automating workflows
- Handling errors
- Generating logs
- Validating pipelines

Conclusion

Python and SQL are usually used together in Data Engineering projects.

12. Python vs PySpark

Use Python For

- Smaller datasets
- APIs
- File handling
- Automation
- Validation
- Configuration
- Utility programs

Use PySpark For

- Very large datasets
- Distributed processing
- Multi-machine execution
- Large-scale transformations

Strong Python fundamentals make PySpark easier to learn.

13. Notebook vs Python Script

Notebook

Useful for:

- Learning
- Experimenting
- Running code step by step
- Exploring data

Python Script

Useful for:

- Automation
- Scheduled execution
- Production pipelines
- Reusable applications

A Python script usually has the `.py` extension.

- `pipeline_validation.py`

Run it using:

- `python pipeline_validation.py`
-

14. Common Mistakes

Unclear Variable Names

Avoid:

- `a = 1000`
- `b = 975`
- `c = a - b`

Prefer:

- `source_count = 1000`
- `target_count = 975`
- `rejected_count = source_count - target_count`

Combining String and Integer Incorrectly

Incorrect:

- `print("Source Count: " + source_count)`

Correct:

- `print("Source Count:", source_count)`

Or:

- `print(f"Source Count: {source_count}")`
-

15. Production Understanding

In practice, values will not usually be hardcoded.

- Source count may come from a file or API.
 - Target count may come from a database.
 - Failed records may be stored separately.
 - Pipeline status may be written to an audit table.
 - Alerts may be triggered when validation fails.
-

Practice Exercise

Create an order-pipeline validation program using:

- Pipeline Name: `daily_order_pipeline`
- Source Name: `order_api`
- Source Records: 2500
- Target Records: 2470

Calculate and display:

- Rejected records
- Pipeline status
- Source count
- Target count

Then change the target count to `2500` and observe the new status.

Additional Challenge

Add:

- `pipeline_owner = "Data Engineering Team"`

Also display:

- Pipeline requires investigation

when the counts do not match.

Interview-Relevant Questions

1. Why is Python used in Data Engineering?

Python is used for file processing, API ingestion, validation, automation, database connectivity, logging, error handling and pipeline development.

2. What is the difference between Python and SQL?

SQL works mainly with data stored inside databases. Python can work with files, APIs, automation, validations and the complete pipeline process.

3. When should Python be used instead of PySpark?

Python is suitable for smaller datasets, APIs, file operations, automation and validation. PySpark is used for large-scale distributed processing.

4. What is record-count validation?

It compares source and target record counts to check whether all records were loaded successfully.

5. How do you calculate rejected records?

- $\text{rejected_count} = \text{source_count} - \text{target_count}$

6. What happens when source and target counts do not match?

The pipeline status can be marked as `VALIDATION_FAILED`, and the pipeline requires investigation.

7. Why are meaningful variable names important?

They make the code easier to understand, debug and maintain.

8. What is the difference between a notebook and a Python script?

A notebook is useful for learning and experimentation. A script is better for automation and production execution.

Quick Revision

- Python handles data and pipeline automation.
- SQL and Python are commonly used together.
- Python is useful for APIs, files and validations.
- PySpark is used for large-scale distributed data.
- Source and target counts help validate pipelines.
- Count mismatch indicates rejected or missing records.
- Scripts are better suited for automation.
- Meaningful variable names improve code quality